GUI

프로그램 작성법

저자	임종우 (E-mail : matlab@matlabschool.com)
편집, 배포	임종관 (E-mail : techuser@oxstart.com)

저작권 : 오정 (http://oxstart.com)

배포처 : MATLAB 전문학교 (http://matlabschool.com)

오정에서 운영하는 홈페이지

오정 웹호스팅	http://oxstart.com
1. 리눅스와 윈도우서버를 이용한 웹호스팅 서비스와 고객을 위한 웹메일서비스 2. .COM, .NET, .ORG, .CC, .JP, .BIZ, .INFO 도메인 등록서비스 3. 국내외 검색사이트 등록서비스 .. (준비중) 4. 비디오테이프영상을 스트리밍파일등 다양한 형태의 컴퓨터용 파일로 변환하는 서비스 5. 이밖에 다양한 무료서비스	
리눅스 서버 : 100메가의 계정과 60메가의 메일계정, 총 160메가 1년 사용료가 7만원 ※ 서비스종류 : 50메가(무료 20메가), 100메가(무료 80메가), 200메가(무료 110메가) 윈도우 서버 : 100메가의 계정과 60메가의 메일계정, 총 160메가 1년 사용료가 12만원 ※ 서비스종류 : 50메가, 100메가, 200메가, 500메가, 1기가	

	http://

한글서당	http://hangulseodang.com
1. 한글97, 나모웹에디터 5, 왕초보(인터넷, 윈도우)에 대한 강의 무료서비스 2. 자료실과 문답(질문/답변), 게시판형태로 운영되는 곳입니다.	

이야기세상	http://ms-book.com
멀티미디어 이야기세상이란 특별한 서비스를 제공하는 곳으로써 다양한 이야기와 다양한 형태의 여러 서비스가 제공되는 곳입니다. 그러나 지금은 준비중이므로 게시판위주로 서비스중입니다.	

MATLAB 전문학교	http://matlabschool.com
임종수의 MATLAB 완벽가이드가 공개된 곳이기도 하며, VHDL, SIMULINK, GUI, ... 등 다양한 강의가 제공되고 있습니다. 동호회형식으로 운영중입니다. 현재 유료 MATLAB 인터넷강의를 준비중입니다.	

이제, slider와 text를 반영한 code를 첨가하면, test.m code는 다음과 같다.

```
%%%%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%%%
function varargout = test(varargin)
% TEST Application M-file for test.fig
%      FIG = TEST launch test GUI.
%      TEST('callback_name', ...) invoke the named callback.
% Last Modified by GUIDE v2.0 17-Jul-2001 19:43:44
if nargin == 0  % LAUNCH GUI
        fig = openfig(mfilename,'reuse');
        % Use system color scheme for figure:
        set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
        % Generate a structure of handles to pass to callbacks, and store it.
        handles = guihandles(fig);

    x_str=get(handles.XEditStr,'string');
    y_str=get(handles.YEditStr,'string');

    x=str2num(x_str);
    y=eval(y_str);
    handles.Line_handle=feval('plot',x,y);
    CurrentLineColor=get(handles.Line_handle,'color');

    set(handles.RSlid,'value',CurrentLineColor(1));
    set(handles.GSlid,'value',CurrentLineColor(2));
    set(handles.BSlid,'value',CurrentLineColor(3));
    set(handles.RSlidValTxt,'string',num2str(CurrentLineColor(1)));
    set(handles.GSlidValTxt,'string',num2str(CurrentLineColor(2)));
    set(handles.BSlidValTxt,'string',num2str(CurrentLineColor(3)));

    guidata(fig, handles);
        if nargout > 0
                varargout{1} = fig;
        end
elseif ischar(varargin{1}) % INVOKE NAMED SUBFUNCTION OR CALLBACK
        try
                [varargout{1:nargout}] = feval(varargin{:}); % FEVAL switchyard
        catch
                disp(lasterr);
        end
```

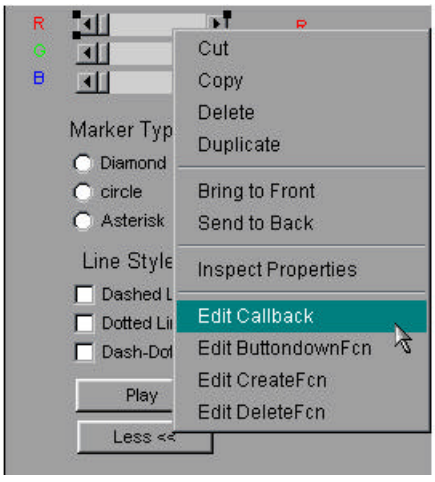
```
end
%%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%
```

위와 같이 code를 수정한 뒤에 다음과 같이 test.m을 실행해 보자.

```
>>test
```

여러분은 slider가 현재 plotting되어 있는 line의 color를 반영하고 있고, slider옆에 있는 text는 현재 slider의 값을 표시하고 있다는 것을 알 수 있을 것이다.

그러나, test.m함수에 있는 어떠한 GUI component를 mouse로 click하여도 아무런 효과가 나타나지 않는다. 이것은 각각의 GUI component를 mouse로 click하는 것과 같은 event를 부여하였을 때, 수행될 **callback 함수**를 지정하지 않았기 때문이다.
그러므로, 이제, 각각의 GUI Component에 event가 발생하였을 때, event를 처리하는 callback함수를 지정하는 방법을 알아보자.
우선, 각각의 slider에 callback함수를 설정하는 방법을 살펴보자.
callback함수를 지정하는 방법은 해당 GUI Component를 mouse로 click하여 선택하고, 오른쪽 mouse 버튼을 누르면, [그림 1-37]과 같이 uicontextmenu가 나타난다.
이때, Edit Callback menu를 선택하면, test.m함수에 다음과 같이 RSlid의 callback함수 prototype이 나타날 것이다.



[그림 1-37] Callback함수 지정 방법.

```
function varargout = RSlid_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.RSlid.
GUIDemo('slider3_Callback',gcbo,[],guidata(gcbo))          ▶ ①
```

여기서, ① line은 삭제한다.

① line은 해석해 보면, RSlid를 click하면, GUIDemo에 있는 slider3_Callback callback함수를 호출하라는 뜻이다. 이때, 입력 매개변수는 gcbo,[],guidata(gcbo)이라는 뜻이다.
특별히, ① line은 UiMenu를 만들 때, 사용할 것인데, 상당히, 중요한 내용을 포함하고 있다.
어쨌든, test.m함수에서 GUIDemo.m함수를 호출하지 않을 것이므로 삭제해야 한다.
그리고, 다음과 같이 삭제된 ① line에 coding을 해준다.

```
CurrentColorVal=get(handles.Line_handle,'color');
RVal=get(h,'value');
set(handles.RSlidValTxt,'string',num2str(RVal));
set(handles.Line_handle,'color',[RVal CurrentColorVal(2:3)])
```

위에 제시한 code는 충분히 여러분이 해석할 수 있을 것이라고 본다.

이제, 저장을 하고 다시 test.m함수를 실행해 보자. 그리고, RSlid를 mouse로 click할 때마다 plotting되어 있는 line의 color가 바뀌는 지, 그리고, 현재 RSlid의 오른쪽에 있는 text가 RSlid의 값을 재대로 반영하고 있는 지 확인해 보자.

나머지, GSlid, BSlid도 같은 방식으로 callback함수를 만들고 다음과 같이 coding해 주면 된다.

```
%%%%%%%%%%%%  test.m Slider의 Callback function Part %%%%%%%%%%%%
% --------------------------------------------------------------------
function varargout = RSlid_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.RSlid.
CurrentColorVal=get(handles.Line_handle,'color');
RVal=get(h,'value');
set(handles.RSlidValTxt,'string',num2str(RVal));
set(handles.Line_handle,'color',[RVal CurrentColorVal(2:3)])
% --------------------------------------------------------------------
function varargout = GSlid_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.GSlid.
CurrentColorVal=get(handles.Line_handle,'color');
GVal=get(h,'value');
set(handles.GSlidValTxt,'string',num2str(GVal));
set(handles.Line_handle,'color',[CurrentColorVal(1) GVal CurrentColorVal(3)])
% --------------------------------------------------------------------
function varargout = BSlid_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.BSlid.
CurrentColorVal=get(handles.Line_handle,'color');
BVal=get(h,'value');
set(handles.BSlidValTxt,'string',num2str(BVal));
```

set(handles.Line_handle,'color',[CurrentColorVal(1:2) BVal])
%%%%%%%%%%% test.m Slider의 Callback function Part %%%%%%%%%%%

이번에는 radiobuttons를 사용하는 방법을 알아보도록 하자.
[그림 1-2]에 나와 있는 radiobuttons의 기능은 plotting되는 line의 marker style을 설정해 주는 역할을 하는데, marker style이란, line의 중간에 첨가되는 symbol을 의미한다.
line의 중간에 첨가 될 수 있는 marker(즉, symbol)의 종류는 [표 1-2]와 같다.

Marker	설명
+	plus sign
o	circle
*	asterisk
.	point
x	cross
s	square
d	diamond
^	upward pointing triangle
v	downward pointing triangle
>	right pointing triangle
<	left pointing triangle
p	five-pointed star (pentagram)
h	six-pointed star (hexagram)
none	no marker (default)

[표 1-6] Line의 marker Style.

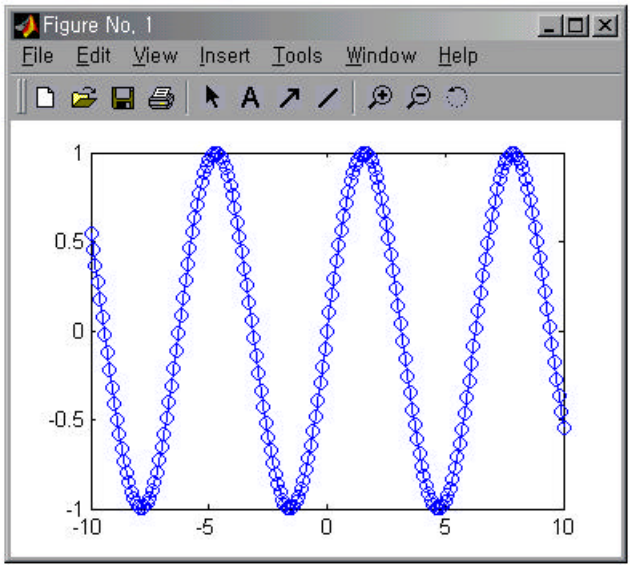
예를 들어서, 다음과 같이 marker style이 무엇인지 살펴보자.

```
>>close all
>>clear all
>>x=-10:0.1:10;
>> y=sin(x);
>> line_hnd=plot(x,y);                          ▶ ①
>> set(line_hnd,'marker','o')                   ▶ ②
>> set(gcf,'color','w')
```

위와 같이 명령하면, [그림 1-38]과 같이 plotting되는 line의 중간 중간에 **o**(즉, circle)이 첨가된 것을 볼 수 있을 것이다.

rm 밖에도 [표 1-2]와 같이 여러 style의 marker들을 line의 중간 중간에 첨가 할 수 있다. 물론, ①번 line과 ②번 line 대신에 다음과 같이 명령하여도 된다.

```
>> plot(x,y,'marker','o')
```



[그림 1-38] line의 marker style을 **o**(circle)로 바꾼 경우.

어쨌든, Diamond radiobutton을 click하면, axes object에 plotting되어 있는 line의 marker style이 diamond로 바뀌도록 radiobutton의 callback함수를 지정해 주어야 한다.
우선, [그림 1-37]과 같이 Diamond radiobutton을 mouse로 click하여 선택하고, mouse의 오른쪽 버튼을 click하여 나타나는 uicontextmenu 중에서 Edit Callback menu를 선택한다.
그러면, test.m에 Diamond radiobutton에 대한 callback함수의 prototype이 다음과 같이 나타날 것이다.

```
function varargout = DMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.DMarkRd.
GUIDemo('radiobutton1_Callback',gcbo,[],guidata(gcbo))          ▶ ①
```

역시, ①번 line은 지워버리고, 다음과 같은 code를 첨가해 보자.

```
set(handles.Line_handle,'marker','diamond')
```

위와 같이 code를 첨가한 뒤에 저장하고 test.m함수를 실행하여 보자.
그리고, Diamond radiobutton을 mouse로 click하여 axes object에 plotting되어 있는 line의 marker style이 바뀌었는지 확인하여 보자.

plotting된 line에 재대로 Diamond marker가 첨가되었으면, 나머지 radiobutton들(즉, circle, Asterisk)에 대해서 같은 방식으로 다음과 같이 code를 첨가해 보자.

```
% --------------------------------------------------------------------
function varargout = DMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.DMarkRd.
set(handles.Line_handle,'marker','diamond')
% --------------------------------------------------------------------
function varargout = CMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.CMarkRd.
set(handles.Line_handle,'marker','o')
% --------------------------------------------------------------------
function varargout = AMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.AMarkRd.
set(handles.Line_handle,'marker','*')
```

그리고, 저장한 뒤에 다시, test.m함수를 실행하여 보자.
각각의 radiobutton을 click할 때마다, 지정한 marker가 line에 첨가되는 것을 확인 할 수 있을 것이다.
그러나, 각각의 radiobutton을 모두 click하면, [그림 1-39]와 같이 모든 radibutton을 선택한 모양을 갖게 되는 것을 볼 수 있을 것이다.



[그림 1-39]

이제, 마지막으로 click한 radiobutton만 선택한 모양을 갖도록 code를 첨가해 보자.
radiobutton뿐만 아니라, checkbox에서도 [그림 1-39]와 같은 현상이 벌어지는 데, 이때에는 다음과 같은 ExChoice.m함수를 사용하면 된다.

```
%%%%%%%%%%%%%%%%%%%  ExChoice.m   %%%%%%%%%%%%%%%%%%%
function ExChoice(state, Check_hnds)
%
%ExChoice.m             이 함수는 배타적 선택이 필요한 경우에 서브 함수로 사용
%                       하면 됩니다.
%
%1999,1,21              By Lim Jong Su
%

state_max=length(Check_hnds);
```

```
if (get(Check_hnds(state),'value') == 1)
    set(Check_hnds([1:(state-1), (state+1):state_max]),'value',0);
else
    set(Check_hnds(state),'value',1);
end
%%%%%%%%%%%%%%%%%%  ExChoice.m  %%%%%%%%%%%%%%%%%%
```

ExChoice.m함수의 입력 매개변수로는 state와 Check_hnds, 2개가 필요한데, state는 사용하는 radiobutton과 checkbox의 index를 의미하며, Check_hnds는 각각의 handle을 모아 놓은 vector를 의미한다.
즉,
test.m함수에서는 3개의 radiobuttons를 사용하고 있는데, 각각의 radiobutton의 handle는 tag name과 같으므로, Diamond radiobutton의 handle은 **DMarkRd**이고, circle radiobutton의 handle은 **CMarkRd**, Asterisk radiobutton의 handle은 **AMarkRd**이 된다. 그러므로,
Check_hnds=[DMarkRd CMarkRd AMarkRd]이 된다.
그리고, Diamond radiobutton의 state는 **1**이고, circle radiobutton의 state는 **2**이고, Asterisk radiobutton의 state는 **3**이 된다.

위와 같은 방식으로 배타적 선택이 필요한 radiobutton과 checkbox를 설정해 주면, 맨 마지막에 선택한 object만 선택한 모양을 갖는다.
이제, test.m함수에 있는 각각의 radiobuttons에 ExChoice를 사용한 배타적 선택 방법을 적용하면, 각각의 radiobuttons에 대한 callback 함수 부분은 다음과 같이 바뀔 것이다.

```
%%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%
% --------------------------------------------------------------------
function varargout = DMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.DMarkRd.
Check_hnds=[handles.DMarkRd handles.CMarkRd handles.AMarkRd];
ExChoice(1,Check_hnds)

set(handles.Line_handle,'marker','diamond')
% --------------------------------------------------------------------
function varargout = CMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.CMarkRd.
Check_hnds=[handles.DMarkRd handles.CMarkRd handles.AMarkRd];
ExChoice(2,Check_hnds)
```

```
set(handles.Line_handle,'marker','o')
% --------------------------------------------------------------------
function varargout = AMarkRd_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.AMarkRd.
Check_hnds=[handles.DMarkRd handles.CMarkRd handles.AMarkRd];
ExChoice(3,Check_hnds)

set(handles.Line_handle,'marker','*')

%%%%%%%%%%%%%  서브 함수   %%%%%%%%%%%%%%%%
function ExChoice(state,Check_hnds)
%
%ExChoice.m              이 함수는 배타적 선택이 필요한 경우에 서브 함수로 사용
%                        하면 됩니다.
%
%1999,1,21               By Lim Jong Su
%
state_max=length(Check_hnds);

if (get(Check_hnds(state),'value') == 1)
    set(Check_hnds([1:(state-1), (state+1):state_max]),'value',0);
else
    set(Check_hnds(state),'value',1);
end
%%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%
```

이제, test.m함수를 저장하고, 실행해 보면, 3개의 radiobutton이 배타적으로 선택되는 것을 확인 할 수 있을 것이다.

ExChoice.m함수가 어떻게 위와 같은 동작을 수행하는 지, 여러분이 직접 ExChoice.m함수를 분석해 보기 바란다.

같은 방식으로, 3개의 Checkbox에 대한 callback함수 부분을 만들어 보자.
여기서, 사용되는 checkbox들은 Line의 style을 지정해 주는 역할을 수행한다.
matlab에서 사용할 수 있는 Line style의 종류는 [표 1-3]과 같다.

Symbol	Line Style
-	solid line (default)
--	dashed line
:	dotted line
-.	dash-dot line
none	no line

[표 1-3] Line style의 종류.

각각의 checkbox에 대한 callback함수의 prototype에 다음과 같은 code를 첨가하면, line style을 제어 할 수 있게 된다.

```
%%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%
% --------------------------------------------------------------------
function varargout = DhLineChk_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.DhLineChk.
Check_hnds=[handles.DhLineChk handles.DtLineChk handles.DDLineChk];
ExChoice(1,Check_hnds)
set(handles.Line_handle,'LineStyle','--')
% --------------------------------------------------------------------
function varargout = DtLineChk_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.DtLineChk.
Check_hnds=[handles.DhLineChk handles.DtLineChk handles.DDLineChk];
ExChoice(2,Check_hnds)
set(handles.Line_handle,'LineStyle',':')
% --------------------------------------------------------------------
function varargout = DDLineChk_Callback(h, eventdata, handles, varargin)
% Stub for Callback of the uicontrol handles.DDLineChk.
Check_hnds=[handles.DhLineChk handles.DtLineChk handles.DDLineChk];
ExChoice(3,Check_hnds)
set(handles.Line_handle,'LineStyle','-.')
%%%%%%%%%%%%%%%%%%%%  test.m  %%%%%%%%%%%%%%%%%%%%
```

지금까지 여러분은 editbox, static text, slider, radiobutton, checkbox component들을 사용하는 방법에 대해서 학습하였다.
이제, ListBox의 사용방법을 알아보도록 하자.
우선, [그림 1-37]과 같이 하여 listbox callback 함수의 prototype을 test.m함수에 첨가해 보자.
앞에서, radiobutton과 checkbox의 사용 방법이 거의 같다는 것을 알 수 있을 것이다.
앞으로, 배울, listbox도 역시, popupmenu와 같은 방식으로 사용하면 된다.

6편에서 계속 ...